

Engenharia de Software – PUC Campinas

Estrutura e Recuperação de Dados I

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Exercício de Árvore:
Avaliação de Expressões Matemáticas (calculadora)

Avaliação de Expressões Matemáticas (calculadora)

- **OBJETIVO:**

- Usar uma árvore binária e recursão para avaliar uma expressão matemática.
- O programa deve calcular o resultado da expressão a partir da árvore gerada automaticamente por um parser que será fornecido que:
 - Lê a expressão digitada pelo usuário (em notação infixa, como $(3 + 5 * 2)$);
 - Verifica se a expressão é válida e bem formada (com parênteses balanceados e operadores corretos);
 - Monta automaticamente a árvore binária de expressão, respeitando a hierarquia dos parênteses e a precedência dos operadores.

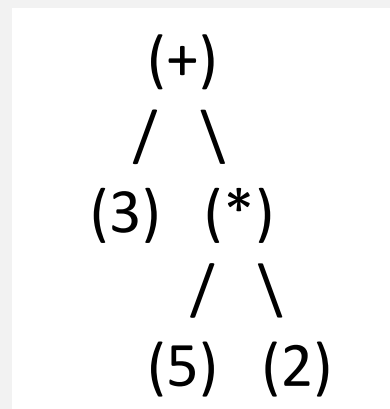
Avaliação de Expressões Matemáticas (calculadora)

○ CONCEITO:

- A árvore de expressão é uma estrutura onde:
 - Cada nó interno contém um operador (+, -, *, /);
 - Cada folha contém um número (um único dígito).
 - A árvore é construída de forma que a ordem dos cálculos siga as regras naturais da matemática (multiplicações e divisões antes de somas e subtrações, e uso de parênteses para alterar a prioridade).

○ EXEMPLO DE ÁRVORE

- Para a expressão $(3 + 5 * 2)$, o parser gera:



Avaliação de Expressões Matemáticas (calculadora)

- TAREFA:

- Implemente a função:

```
int evaluateExpression(Node *currentNode);
```

- Essa função deve calcular o resultado da expressão representada pela árvore binária.

- LÓGICA DO ALGORITMO

- O raciocínio é recursivo:
 - Comece pela raiz da árvore.
 - Se o nó for uma folha, o valor do nó é retornado (é um número).
 - Se o nó contiver um operador (+, -, *, /):
 - Chame recursivamente a função para o filho esquerdo e o filho direito;
 - Aplique o operador aos valores retornados por essas chamadas;
 - Retorne o resultado.

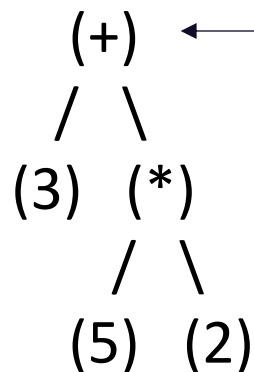
Avaliação de Expressões Matemáticas (calculadora)

○ OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da raiz, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

○ EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



`evaluateExpression(root)` → é operador **+**
→ `evaluateExpression(left)` **+** `evaluateExpression(right)`

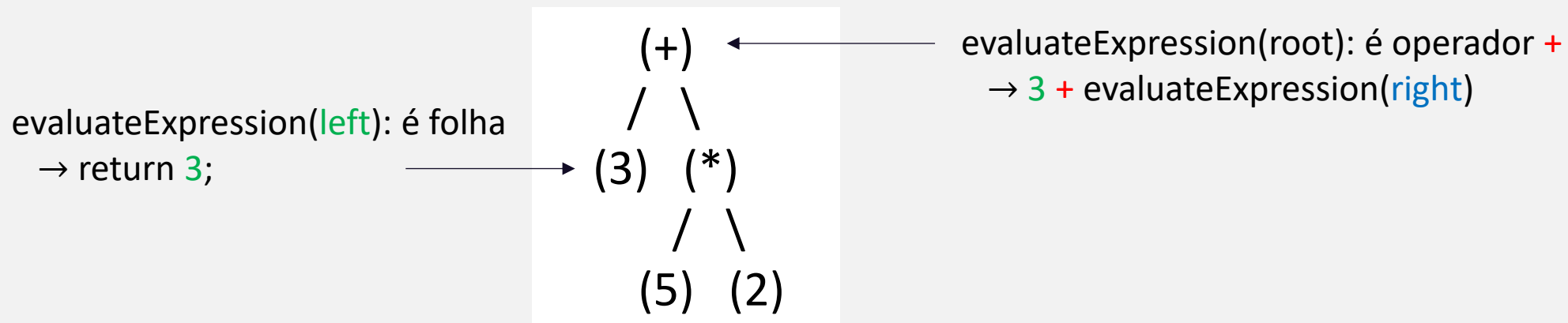
Avaliação de Expressões Matemáticas (calculadora)

○ OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da raiz, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

○ EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



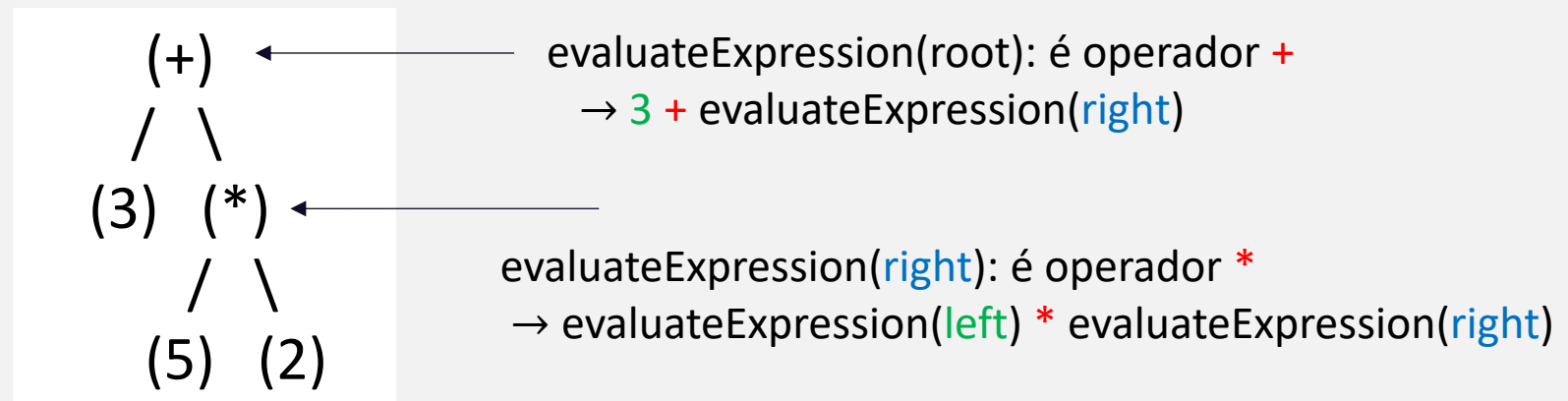
Avaliação de Expressões Matemáticas (calculadora)

○ OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da raiz, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

○ EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



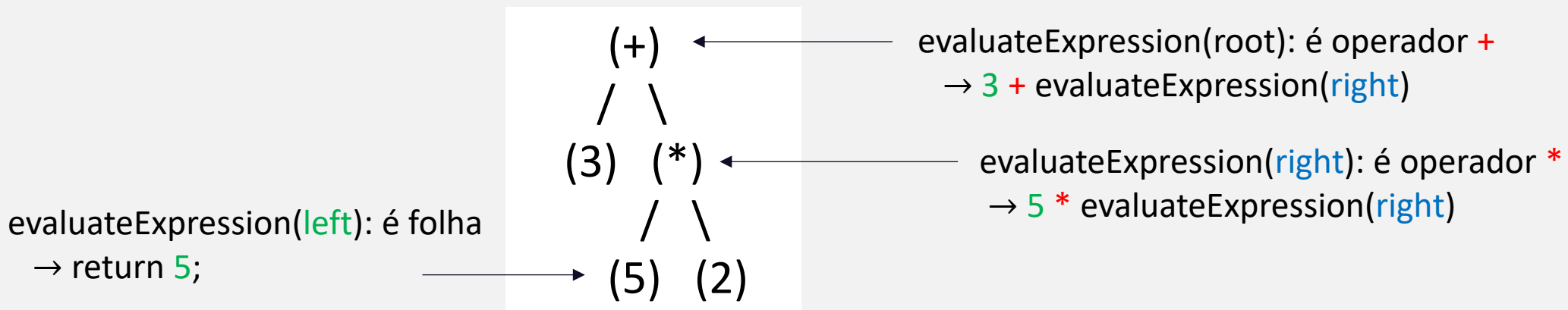
Avaliação de Expressões Matemáticas (calculadora)

○ OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da raiz, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

○ EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



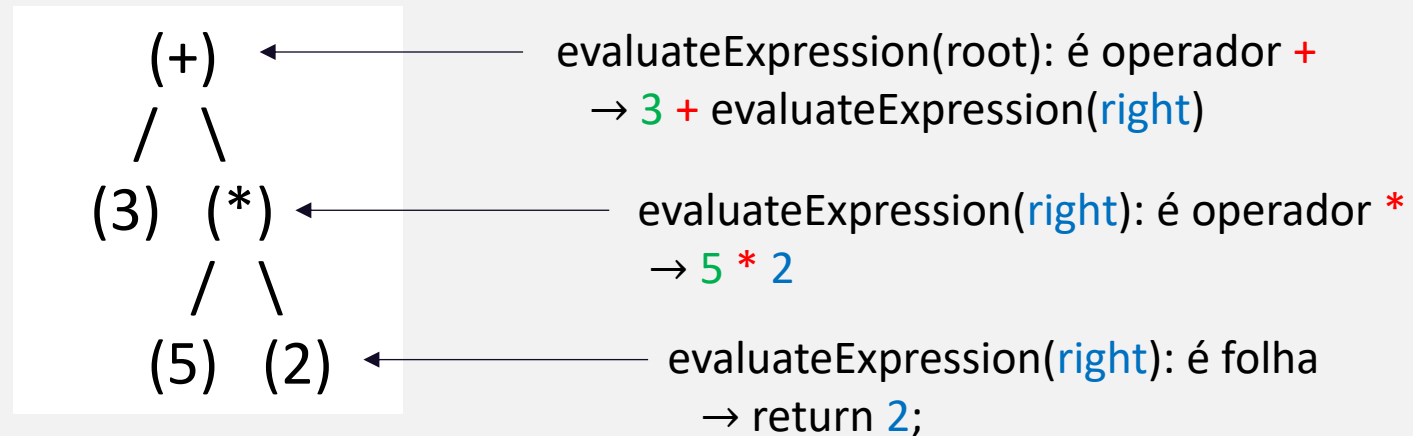
Avaliação de Expressões Matemáticas (calculadora)

○ OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da raiz, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

○ EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



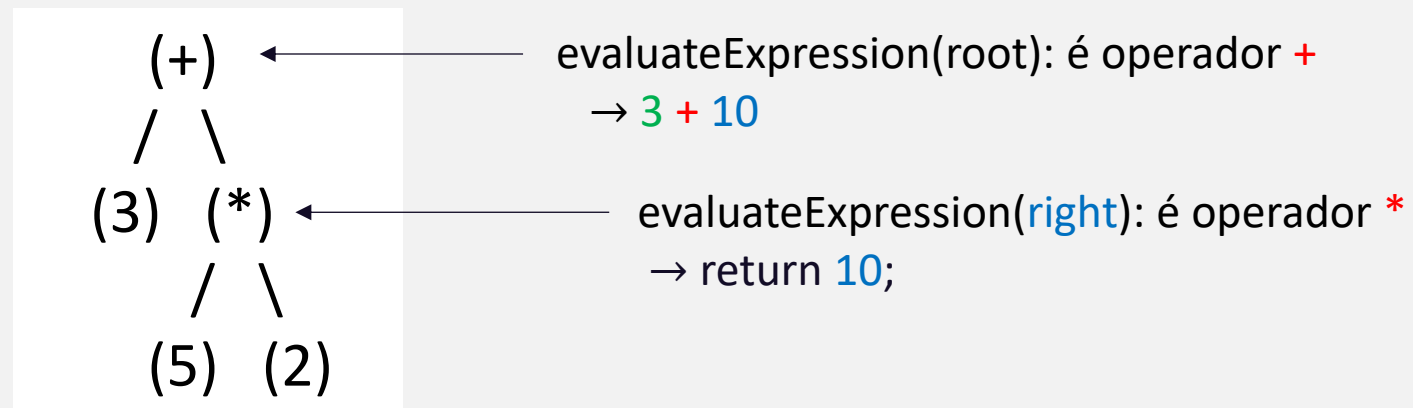
Avaliação de Expressões Matemáticas (calculadora)

○ OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da raiz, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

○ EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



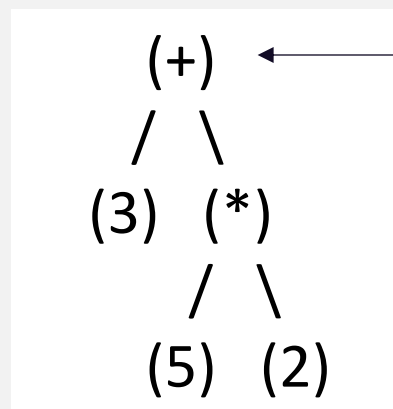
Avaliação de Expressões Matemáticas (calculadora)

○ OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da raiz, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

○ EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



← `evaluateExpression(root)`: é operador `+`
→ `return 13;`

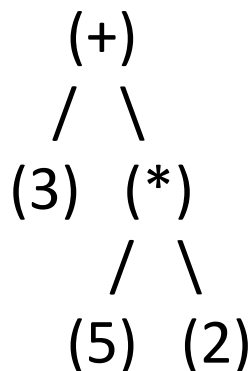
Avaliação de Expressões Matemáticas (calculadora)

- OBSERVAÇÃO IMPORTANTE:

- A função `evaluateExpression` é chamada a partir da **raiz**, mas quem “decide” ir até as folhas é a própria recursão.
- A recursão mergulha até o último nível (as folhas) e sobe resolvendo as operações passo a passo.

- EXEMPLO DE EXECUÇÃO

- Para a expressão $(3 + 5 * 2)$:



Resposta: 13

Avaliação de Expressões Matemáticas (calculadora)

- AQUI ESTÁ O SEGREDO!!!!!!!!!!!!!!

- Comece pela raiz da árvore.
- Se o nó for uma folha, o valor do nó é retornado (é um número).
- Se o valor do nó contiver um operador (+, -, *, /):
 - Chame recursivamente a função para o filho esquerdo e o filho direito;
 - Aplique o operador aos valores retornados por essas chamadas;
 - Retorne o resultado.

- ESTRUTURA DO NÓ

```
typedef struct Node {  
    char data;           // se operador  
    double value;        // se número  
    struct Node *left;   // filho esquerdo  
    struct Node *right;  // filho direito  
} Node;
```

Avaliação de Expressões Matemáticas (calculadora)

- CÓDIGO

- O código a seguir inclui:

- Verificação de consistência da expressão (pilha de delimitadores);
 - Parser que insere a expressão numa árvore.
 - A implementação de vocês deverá ser feita dentro do escopo da função:

```
double evaluateExpression(Node *currentNode)
```